TP 9 – Météo en direct

Consommation d'API avancée – recherche différée, appels combinés, secrets

Pr. Ahmed Bentajer

Objectifs

- Différer une recherche déclenchée à chaque frappe (*debounce*)
- Enchaîner et paralléliser plusieurs appels `fetch` liés entre eux
- Distinguer plusieurs types d'erreurs réseau et les afficher clairement
- Stocker une clé API sans jamais la faire apparaître dans le code source

Cahier des charges

Une petite application météo à partir de l'API publique OpenWeatherMap : on cherche une ville, on choisit la bonne parmi les suggestions (plusieurs villes peuvent porter le même nom), et on affiche sa météo actuelle ainsi que ses prévisions regroupées par jour.

Le gabarit (`App.vue`) est entièrement câblé côté affichage – vous ne touchez pas au `<template>`. Tout le travail est dans les méthodes et la `computed` du `<script>`, réparties en 6 TODO. Pas de routeur ni de composants séparés dans ce TP : on se concentre sur la logique réseau.

`npm install && npm run dev` pour démarrer – mais lisez d'abord la section suivante, l'application ne fonctionnera pas sans une clé API valide.

1 Mise en route – obtenir et stocker votre clé API

OpenWeatherMap est gratuit mais nécessite un compte et une clé personnelle. Une clé API, c'est un secret : si vous la mettez en clair dans un fichier `.js` et que vous poussez ce fichier sur GitHub, n'importe qui peut la récupérer et l'utiliser à votre place – jusqu'à épuiser votre quota, voire pire selon le service.

1. Créez un compte gratuit sur home.openweathermap.org/users/sign_up, puis récupérez votre clé dans l'onglet « API keys » de votre compte (elle peut mettre quelques minutes à s'activer après la création).

2. À la racine du projet (à côté de `package.json`), dupliquez le fichier `.env.example` en un nouveau fichier nommé `.env`, et remplacez la valeur d'exemple par votre vraie clé.

3. Ce fichier `.env` est volontairement listé dans `.gitignore` : il reste sur votre machine et n'est **jamais** envoyé sur un dépôt Git. C'est `.env.example` (sans vraie clé) qui, lui, est suivi par Git et sert de modèle à toute personne qui récupère le projet.

4. Dans le code, Vite expose automatiquement au navigateur toute variable de `.env` dont le nom commence par `VITE_`, via `import.meta.env`. C'est déjà écrit en haut de `App.vue` :

```
const API_KEY = import.meta.env.VITE_OWM_API_KEY
```

5. Après avoir créé ou modifié `.env`, redémarrez `npm run dev` (Vite ne relit ce fichier qu'au démarrage).

Pour info

C'est exactement la méthode utilisée en entreprise pour tous les secrets (clés API, mots de passe de base de données...) : un fichier `.env` local et ignoré par Git, et un `.env.example` qui documente les variables attendues sans jamais contenir de vraie valeur. Une clé qui finit malgré tout sur GitHub doit être considérée comme compromise et régénérée immédiatement, même après l'avoir supprimée du dépôt – l'historique Git la garde.

2 Recherche différée (TODO 1)

Complétez `onInputRecherche()`, appelée à chaque frappe dans le champ de recherche. Elle ne doit pas lancer un appel réseau à chaque lettre tapée, mais attendre que l'utilisateur marque une pause d'environ 400ms.

Piège classique

Un *debounce* mal écrit programme un nouveau `setTimeout` à chaque frappe **sans annuler le précédent** : si vous tapez « Saf » puis « i », deux requêtes partent, et rien ne garantit que la réponse à « Safi » arrive après celle à « Saf ». Résultat : l'écran peut afficher les suggestions de la recherche la *plus lente*, pas de la plus récente. La parade : toujours appeler `clearTimeout(this.debounceTimer)` en tout début de fonction, avant de reprogrammer un nouveau minuteur.

3 Géocodage : retrouver une ville (TODO 2)

Complétez `rechercherVilles()` : elle interroge l'API de géocodage et remplit `this.suggestions`. Trois cas d'erreur sont à distinguer dans `erreurSuggestions` : clé invalide (`401`), quota dépassé (`429`), et ville introuvable – ce dernier cas n'est **pas** une erreur HTTP.

Piège classique

Contrairement à une ville qui n'existe pas dans REST Countries (TP7), une recherche géocodage sans résultat ne renvoie pas une erreur `404` : l'API répond `200 OK` avec un tableau vide `[]`. Si vous ne testez que `reponse.ok`, une faute de frappe dans le nom de la ville passera silencieusement, avec une liste de suggestions vide et aucun message pour l'utilisateur.

4 Sélection et historique (TODO 3)

Complétez `choisirVille(ville)`, appelée au clic sur une suggestion *ou* sur un élément de l'historique. Le paramètre `ville` a la même forme dans les deux cas (c'est un objet de `suggestions` ou d'`historique`), ce qui simplifie beaucoup cette méthode.

5 Météo actuelle et prévisions, en parallèle (TODO 4)

Complétez `chargerMeteo()`. Les deux appels (météo actuelle, prévisions) ne dépendent pas l'un de l'autre : les lancer l'un après l'autre avec deux `await` successifs fonctionnerait, mais ferait attendre le second inutilement après le premier. `Promise.all` les lance en même temps et attend que les deux soient terminés.

6 Changer d'unité (TODO 5)

Complétez `basculerUnite()`. Contrairement à un simple changement d'affichage, les températures viennent *déjà converties* par l'API selon le paramètre `units=` – changer d'unité veut donc dire refaire

l'appel, pas juste reformater un nombre côté client.

7 Regrouper les prévisions par jour (TODO 6)

L'API `/forecast` renvoie une prévision toutes les 3 heures sur 5 jours (environ 40 entrées), chacune avec un champ `dt_txt` du type `"2024-05-12 15:00:00"`. Complétez la `computed` `previsionsParJour` pour regrouper ces tranches en une seule ligne par jour, avec le minimum et le maximum de température observés ce jour-là.

Pour info

Ce n'est pas une notion Vue à proprement parler, mais un exercice classique de manipulation de tableau que vous retrouverez dans énormément de traitements de données réels : construire un objet intermédiaire indexé par clé (ici la date), puis le transformer en tableau avec `Object.values(...)`.

Bonus

- Au montage de l'app, utilisez `navigator.geolocation.getCurrentPosition(...)` pour proposer automatiquement la météo de la position actuelle (avec les coordonnées, vous pouvez appeler directement `chargerMeteo()` sans passer par le géocodage).
- Persistez `historique` dans `localStorage` pour qu'il survive au rechargement de la page (`JSON.stringify`/`JSON`

TP indépendant sur la consommation d'API – pour l'exercice de mise en page avancée, voir le TP 10 (clone d'interface).